МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Отчёт по лабораторной работе №3
по курсу «Операционные системы и архитектура компьютера»

Выполнил: В. Е. Авраменко
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 20.10.2026
Оценка: _____

Москва, 2026

Цель и задание

Цель лабораторной работы №3 – освоение создания и синхронизации потоков средствами операционной системы. Выполняется вариант 13: наложить K раз фильтр с матрицей свёртки на матрицу вещественных чисел; размер окна задаёт пользователь.

Программа написана на C11 для Linux, потоки создаются через POSIX pthread. Пользователь вводит размеры и элементы матрицы, размер и коэффициенты квадратного ядра и количество проходов K. Ключ `--threads P` задаёт максимальное число одновременно существующих потоков приложения, включая главный. Требуются демонстрация потоков средствами ОС и исследование ускорения и эффективности.

Математическая модель

Для матрицы A размера R на C и ядра H размера M на M один проход вычисляет

$$B_{i,j} = \sum_{u=0}^{M-1} \sum_{v=0}^{M-1} H_{u,v}\, A_{i+c-u,\; j+c-v}, \qquad c = (M-1)/2.$$

Размер результата сохраняется. За границами A значения считаются нулевыми. M положительно и нечётно, поэтому центральный элемент определён однозначно. Ядро переворачивается по обеим осям: это свёртка, а не корреляция. Коэффициенты не нормализуются автоматически. Ядро может быть больше матрицы; вне пересечения с ней используются нули. Эти правила уточняют детали, не определённые в исходном задании.

После каждого прохода B становится входом следующего. Всего выполняется K проходов. Для хранения данных используются два буфера матрицы и один буфер ядра. Оценка числа операций: $O(KRCM^2)$; объём численных данных: $(2RC + M^2)$ sizeof(double). Дополнительно нужны $O(P)$ записей заданий и ресурсы стеков потоков, выделяемые ОС.

Распараллеливание и синхронизация

Фактический плановый лимит $P = \min(P_{\text{requested}}, R)$. Строки делятся на P непрерывных диапазонов. Размер базового диапазона равен $\lfloor R/P \rfloor$; первые $R \bmod P$ диапазонов получают по одной дополнительной строке. Рабочие потоки обрабатывают P-1 диапазонов, главный – оставшийся диапазон. При P=1 новые потоки не создаются.

В пределах прохода входная матрица и ядро доступны только для чтения; каждый поток записывает только собственные строки результата. Мьютекс для элементов матрицы не нужен: записываемые диапазоны не пересекаются. После вычисления своего диапазона главный поток вызывает pthread_join для всех запущенных работников. Только после их завершения буферы меняются ролями. Таким образом, следующий проход не начинает чтение незавершённого результата.

Работники создаются заново на каждом проходе. Это упрощает запуск, остановку и обработку ошибок; стоимость создания и ожидания входит в измеренное время. Постоянный пул потоков для этой учебной реализации не вводится. Число созданий за всю работу может превышать P, но одновременно существует не более P потоков приложения.

Структура и читаемость программы

Таблица 1 – Модули программы

Файл	Ответственность
`matrix.c/.h`	Представление матрицы, безопасное выделение памяти, свёртка диапазона строк.
`filter.c/.h`	Запуск работников, ожидание, распределение строк и K проходов.
`console.c/.h`	Параметры запуска, построчный ввод с проверкой, подсказки и вывод.
`main.c`	Соединение модулей, измерение времени, освобождение матриц и код завершения.

Используется обычная модульная структура C без дополнительных архитектурных слоёв и указателей на единственную реализацию. main занимает 20 физических строк, самая длинная C-функция – 33 строки. Все C-функции и функции вспомогательных Python-скриптов не превышают 40 строк. Ограничения длины и оформления проверены при подготовке работы. Переходов goto в C-коде нет.

run_rows вычисляет диапазон, start_workers создаёт потоки и назначает строки, join_workers дожидается завершения, run_pass объединяет один проход. apply_filter владеет вспомогательной матрицей и меняет буферы. read_problem владеет буфером строки ввода; read_input читает данные с ранним возвратом при ошибке. main освобождает обе матрицы при любом обычном исходе.

Проверка ввода и ошибок

Размеры, K и лимит потоков должны быть положительными целыми числами. До умножения размеров проверяется переполнение; число байтов матрицы ограничено PTRDIFF_MAX. Это также обеспечивает представимость знаковой арифметики индексов при обработке краёв. Координаты приводятся к ptrdiff_t до вычитания смещения.

Числа читаются как double через strtod. В текущей версии модуль вводимых элементов матрицы и ядра дополнительно ограничен ULLONG_MAX; проверяется значение после округления в double. NaN, Inf и переполнение ввода отклоняются. Представимые субнормальные числа принимаются; ненулевые значения, слишком малые для double и округляющиеся в ноль при вводе, отклоняются. Если в ходе свёртки получен неконечный результат, проход завершается ошибкой, все созданные работники ожидаются, матрица результата не выводится.

В терминале неверная строка вводится повторно целиком, остальные принятые строки сохраняются. getline поглощает всю строку и исключает зацикливание на ошибочном токене. При перенаправлении ввода неверные данные немедленно дают ненулевой код завершения. Недостаточный ввод и системные ошибки сопровождаются диагностикой в stderr.

Если pthread_create не смог создать очередного работника, программа сначала ожидает уже запущенных. При ошибке pthread_join процесс немедленно завершается через _Exit: память не освобождается, пока неизвестно, закончил ли поток обращаться к ней. Такая ветка проверяется искусственным отказом. Ошибки печати обрабатываются; SIGPIPE игнорируется, чтобы разрыв выходного канала был доступен как ошибка записи.

Сборка и пример использования

Из каталога проекта в WSL:

```
cd /mnt/c/Projects/os-labs/lab3
make
./convolution --threads 4
```

Программа последовательно запрашивает размеры, строки матрицы, нечётную сторону ядра, строки ядра и K. Дробная часть задаётся точкой; значения строки разделяются пробелами. Для завершения незаконченного ввода можно нажать Ctrl+D. Справка: `./convolution --help`. По умолчанию лимит равен одному потоку.

Один формат используется и в терминале, и в файле. Подсказки появляются только при терминальном stdin. Результат в stdout содержит только числа; PID, лимит и время выводятся отдельно в stderr.

Листинг 1 – Вход: единичное ядро и два прохода

```
2 3
1 2 3
4 5 6
3
0 0 0
0 1 0
0 0 0
2
```

Листинг 2 – Фактический результат запуска

```
1 2 3
4 5 6
```

```
./convolution --threads 4 < input.txt
make -C report
```

Демонстрация числа потоков

Для наблюдения используется стандартная команда Linux:

```
ps -L -p PID -o pid,tid,nlwp,stat,comm
```

PID программа печатает перед расчётом. При подготовке работы отдельный проверочный сценарий запускал длительную обработку и периодически вызывал ps. При лимитах 1, 2 и 4 наблюдались соответственно 1, 2 и 4 потока. Для матрицы из двух строк при запрошенном лимите 8 наблюдались 2 потока. Столбец NLWP показывает число потоков, TID – идентификатор каждого из них.

Листинг 3 – Снимки ps и проверка strace

```
rows=512 requested=1 peak=1
    PID     TID NLWP STAT COMMAND
  16172   16172    1 R+   convolution
rows=512 requested=2 peak=2
    PID     TID NLWP STAT COMMAND
  16182   16182    2 Rl+  convolution
```

```
  16182   16184    2 Rl+  convolution
rows=512 requested=4 peak=4
    PID     TID NLWP STAT COMMAND
  16202   16202    4 Rl+  convolution
  16202   16204    4 Rl+  convolution
  16202   16205    4 Rl+  convolution
  16202   16206    4 Rl+  convolution
rows=2 requested=8 peak=2
    PID     TID NLWP STAT COMMAND
  16245   16245    2 Rl+  convolution
  16245   16259    2 Rl+  convolution
strace: 6 successful worker creations for 2 passes at P=4
```

Это наблюдённый пик при периодических снимках, а не непрерывный мониторинг каждого мгновения. Верхняя граница также следует из кода: на каждом проходе создаётся не более P-1 работников, и перед новым проходом они все ожидаются. В отдельном запуске strace -f для двух проходов и P=4 зарегистрировал шесть успешных созданий работников, то есть по три на проход. Трасса сохранена в inc/threads-trace.txt.

Полная трасса: создание, ожидание и завершение потоков

Дополнительно 29.10.2026 снята полная трасса без фильтра системных вызовов. Вход: матрица 5×65536 из единиц, ядро 31×31 с единицей в центре и нулями в остальных позициях, K=2, лимит P=4. Единичное ядро сохраняет матрицу; фактический результат совпал с входной матрицей, код завершения равен нулю. Команда выполняется из корня проекта:

Листинг 4 – Команда полной трассировки

```
strace -f -tt -T -o /tmp/lab3-strace-full.txt \
  ./convolution --threads 4 \
  < report/inc/strace-full-input.txt \
  > /tmp/lab3-result.txt 2> /tmp/lab3-diagnostic.txt
```

Ключ `-f` отслеживает работников, `-tt` добавляет время вызова, `-T` – длительность системного вызова, `-o` сохраняет трассу отдельно. Фильтр `-e` не используется: записываются также чтение, выделение памяти, сигнальные операции, вывод и завершение. Стандартное сокращение содержимого длинных буферов strace сохраняется. Исходная полная трасса из 481 строки приложена в `inc/strace-full.txt`, вход – в `inc/strace-full-input.txt`; параметры и SHA-256 исходников – в `inc/strace-full-checks.json`. Следующий листинг содержит выбранные строки без изменения их содержимого, с номерами из полного файла.

Листинг 5 – Фрагмент полной трассы с ожиданием работников

```
209: 4989  14:16:41.329965
↪  clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY⌋
↪  SVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
↪  child_tid=0x78bc1e6fd990, parent_tid=0x78bc1e6fd990, exit_signal=0,
↪  stack=0x78bc1defd000, stack_size=0x7fff80, tls=0x78bc1e6fd6c0} =>
↪  {parent_tid=[4992]}, 88) = 4992 <0.000135>
247: 4989  14:16:41.341125 futex(0x78bc1e6fd990,
↪  FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 4992, NULL, FUTEX_BITSET_MATCH_ANY
↪  <unfinished ...>
259: 4992  14:16:41.344817 exit(0 <unfinished ...>
```

```
263: 4989  14:16:41.345516 <... futex resumed>) = 0 <0.004055>
264: 4992  14:16:41.345745 +++ exited with 0 +++
266: 4989  14:16:41.346081
↪  clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY
↪  SVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
↪  child_tid=0x78bc1d6fb990, parent_tid=0x78bc1d6fb990, exit_signal=0,
↪  stack=0x78bc1cefb000, stack_size=0x7fff80, tls=0x78bc1d6fb6c0} =>
↪  {parent_tid=[4995]}, 88) = 4995 <0.000192>
300: 4989  14:16:41.355528 futex(0x78bc1d6fb990,
↪  FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 4995, NULL, FUTEX_BITSET_MATCH_ANY
↪  <unfinished ...>
312: 4995  14:16:41.359057 exit(0)           = ?
313: 4989  14:16:41.359369 <... futex resumed>) = 0 <0.003538>
314: 4995  14:16:41.359495 +++ exited with 0 +++
481: 4989  14:16:41.690882 +++ exited with 0 +++
```

Главный поток имеет PID/TID 4989. На первом проходе созданы работники 4992, 4993 и 4994; на втором – 4995, 4996 и 4997. Пять строк делятся как 2+1+1+1: первый работник получает две строки, остальные работники и главный – по одной. Шесть созданий за два прохода не нарушают лимит: на каждом проходе существует не более трёх работников и главного потока.

Строка 209 показывает успешный `clone3`, вызванный библиотекой при `pthread_create`: результат 4992 – TID работника; `CLONE_THREAD` обозначает поток того же процесса, `CLONE_VM` – общую память. В поле `child_tid` указан адрес `0x78bc1e6fd990`.

В строке 247 главный поток вызывает `futex` с операцией ожидания именно по этому адресу и ожидаемым значением 4992. Совпадение адреса и TID связывает ожидание с завершением работника внутри `pthread_join`, а не с произвольной блокировкой. Работник вызывает `exit(0)` в строке 259; в строке 263 ожидание главного завершается успешно после примерно 4.055 мс. Сообщение о завершении появляется в строке 264; порядок печати сообщений strace не задаёт строгого порядка событий разных потоков.

В строке 266 начинается создание работников следующего прохода. Аналогичная пара на втором проходе: ожидание TID 4995 в строке 300, его завершение в строке 312 и возврат из `futex` в строке 313 после примерно 3.538 мс. Строка 481 показывает успешное завершение главного потока. Для остальных работников блокирующий вызов не потребовался: к моменту их `join` они уже завершились. Поэтому число вызовов `pthread_join` не обязано совпадать с числом видимых ожиданий `futex`.

Трасса не показывает арифметику и вход в `run_rows`. Времена этого запуска искажены накладными расходами strace и не используются для исследования ускорения.

Исследование ускорения и эффективности

Измерения проведены 20.10.2026 в WSL2, Linux 6.6.87.2, на Intel Core Ultra 5 225H; ОС предоставляет 14 логических процессоров. Компилятор GCC 13.3.0, ключи -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror, библиотеки pthread и libm. Подробности окружения сохранены в inc/machine.json.

Проверены квадратные матрицы 128, 512 и 1024, ядро 5 на 5 с коэффициентами $1/25$ и K=20. Строки входной матрицы состоят из повторяющихся целых значений от -8 до 8. Для каждого размера и P из 1, 2, 4, 8 выполнен один прогрев и пять измерений; порядок P

перемешивался детерминированно на каждом повторе. Для всех P проверено полное совпадение SHA256 вывода на одинаковом входе. Математическую корректность отдельно проверяют интеграционные тесты.

Время определяется CLOCK_MONOTONIC непосредственно вокруг apply_filter. В него входят выделение вспомогательной матрицы, вычисления, создание и ожидание потоков. Чтение данных, печать результата и проверка хеша в него не входят. В таблице приведены медиана пяти повторов и диапазон минимума–максимума, в миллисекундах.

$$S_P = T_1/T_P, \qquad E_P = S_P/P.$$

Здесь P включает главный поток, который также вычисляет свой диапазон строк.

Таблица 2 – Время, ускорение и эффективность: ядро 5 на 5, K=20

N	P	Медиана, мс	Мин.–макс., мс	S_P	E_P
128	1	4.426	4.258–4.887	1.000	1.000
128	2	6.388	4.892–8.052	0.693	0.346
128	4	5.266	4.292–6.604	0.841	0.210
128	8	12.011	11.210–13.693	0.369	0.046
512	1	85.485	75.769–96.504	1.000	1.000
512	2	46.361	39.109–50.759	1.844	0.922
512	4	36.125	31.674–42.923	2.366	0.592
512	8	30.133	27.994–44.572	2.837	0.355
1024	1	367.781	354.994–391.327	1.000	1.000
1024	2	182.731	158.955–258.864	2.013	1.006
1024	4	104.670	95.675–115.829	3.514	0.878
1024	8	86.599	83.261–102.258	4.247	0.531

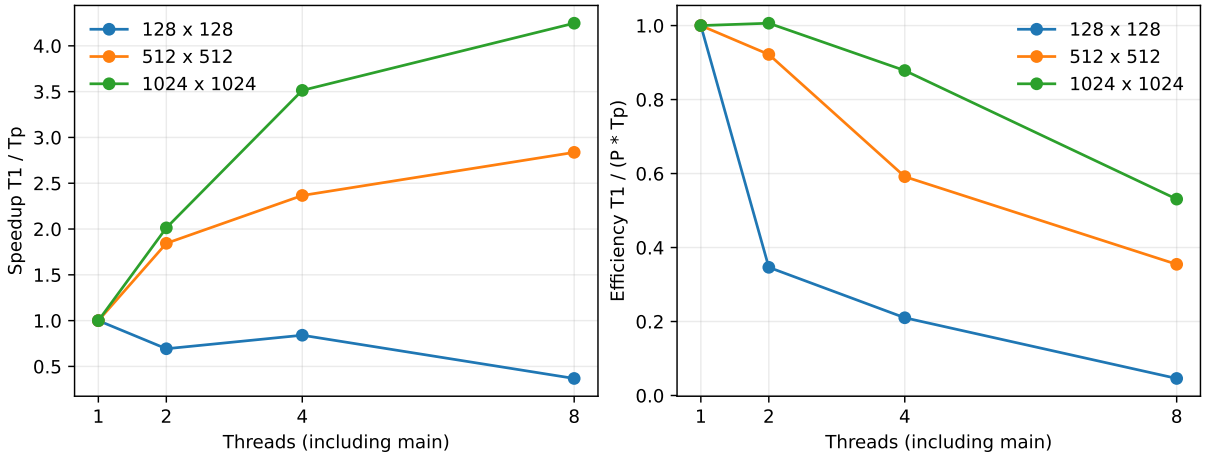


Рисунок 1 – Ускорение и эффективность по медианам пяти повторов

Для 128 на 128 добавление потоков замедляет работу: объём вычислений недостаточен, чтобы компенсировать запуск работников и ожидание на каждом из 20 проходов. Для 512 на 512 и 1024 на 1024 рост объёма вычислений улучшает соотношение полезной работы и накладных расходов; восемь потоков дают ускорение примерно 2.84 и 4.25 соответственно.

Эффективность на восьми потоках составляет примерно 0.355 и 0.531 для двух больших размеров. Линейного ускорения нет: в измеренное время входят последовательные участки, распределение работы, создание потоков и ожидание наиболее медленного диапазона. Разброс может быть связан с планировщиком, конкуренцией за ресурсы и изменением частоты процессора; отдельные вклады этих причин не измерялись.

Для 1024 и P=2 получено $E_P \approx 1.006$. Превышение единицы меньше одного процента и не доказывает сверхлинейное масштабирование: отношение медиан подвержено шуму измерений, что видно по диапазонам времени. Выводы относятся к этой машине, нагрузке и простому способу запуска потоков; универсальное ускорение не утверждается. Пилотные короткие запуски сохранены отдельно под префиксом pilot и в итоговой таблице не смешиваются с K=20.

Проверки и независимое ревью

Принудительная первоначальная сборка и последующие пересборки изменённых исходников прошли с указанными строгими флагами GCC. При подготовке работы успешно выполнены 14 именованных групп проверок с дополнительными вариантами входов. Проверены единичное и асимметричное ядро, заранее известный ответ для сдвига, дробные и отрицательные значения, ядро больше матрицы, несколько проходов, неравные диапазоны строк, ограничение потоков, неверный и неполный ввод, субнормальные числа, переполнение вычислений и ошибка вывода.

Интерактивная проверка использует настоящий псевдотерминал и подтверждает повтор неверных строк. Отказы pthread_create и pthread_join проверяются отдельной сборкой с подменой этих функций через линкер. ASan и UBSan успешно выполнили те же 14 групп для основной программы; сценарии подмены pthread при этом используют отдельный обычный тестовый исполняемый файл. Санитайзеры не заменяют доказательство отсутствия гонок: оно основано на непересекающихся записях, неизменяемом входе и join перед сменой буферов.

Текстовые протоколы находятся в inc/checks.txt и inc/style.txt. Фактический вывод примера совпал с ожидаемым результатом побайтово. Независимое ревью проверяет декомпозицию и владение ресурсами после реализации; заключение сохраняется в inc/review.txt. После завершения проверок вспомогательные тестовые скрипты удалены из сдаваемого проекта; сохранены протоколы и результаты измерений. Проверка охватывает C11 и POSIX API, но не заявляет соответствия всем стандартам C++: язык задания – C.

Повторная проверка 21.10.2026 после разделения условий разбора чисел: строгая сборка и 63 запуска прошли успешно. Результаты свёртки сопоставлены с независимым эталоном; проверены ошибочный ввод и отказ вывода. Все 12 строк итоговой таблицы повторно вычислены из 60 сохранённых измерений и совпали с отчётом. Бенчмарк повторно не запускался, новых утверждений об ускорении не делается.

Выводы

Реализована свёртка пользовательской матрицы с заданным ядром, K проходами и ограничением числа потоков. Между проходами выполняется ожидание работников. Корректность проверена эталонными и граничными примерами; управление памятью – санитайзерами; потоки – средствами ОС и трассировкой. Измерения показывают ускорение на больших матрицах и преобладание накладных расходов на малых.

Приложение А. Исходный код

Листинг А.1 – Файл main.c

```
#include "console.h"
#include "filter.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

static int ignore_sigpipe(void) {
    struct sigaction action = {.sa_handler = SIG_IGN};
    if (sigemptyset(&action.sa_mask) == -1 || sigaction(SIGPIPE, &action,
    ↪  NULL) == -1) {
        perror("sigaction");
        return -1;
    }
    return 0;
}

static double monotonic_seconds(void) {
    struct timespec now;
    if (clock_gettime(CLOCK_MONOTONIC, &now) == -1) {
        perror("clock_gettime");
        return -1.0;
    }
    return (double)now.tv_sec + (double)now.tv_nsec / 1e9;
}

static int process_problem(Matrix *matrix, const Matrix *kernel, size_t
↪  iterations, size_t limit) {
    size_t threads = limit < matrix->rows ? limit : matrix->rows;
    fprintf(stderr, "pid=%ld thread_limit=%zu threads=%zu\n", (long)getpid(),
    ↪  limit, threads);
    double start = monotonic_seconds();
    if (start < 0) {
        return -1;
    }
    if (apply_filter(matrix, kernel, iterations, threads) == -1) {
        perror("Фильтрация");
        return -1;
    }
    double finish = monotonic_seconds();
    if (finish < 0) {
        return -1;
    }
    double elapsed = finish - start;
    fprintf(stderr, "elapsed_seconds=%.9f\n", elapsed);
    if (print_matrix(matrix) == -1) {
        perror("Вывод матрицы");
        return -1;
```

```
    }
    return 0;
}

int main(int argc, char **argv) {
    size_t limit;
    int option = parse_options(argc, argv, &limit);
    if (option != 0) {
        return option > 0 ? EXIT_SUCCESS : EXIT_FAILURE;
    }
    if (ignore_sigpipe() == -1) {
        return EXIT_FAILURE;
    }

    Matrix matrix = {0}, kernel = {0};
    size_t iterations = 0;
    int result = read_problem(&matrix, &kernel, &iterations);
    if (result == 0) {
        result = process_problem(&matrix, &kernel, iterations, limit);
    }
    free(matrix.data);
    free(kernel.data);
    return result == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Листинг А.2 – Файл matrix.h

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stddef.h>

typedef struct {
    size_t rows;
    size_t cols;
    double *data;
} Matrix;

int matrix_create(Matrix *matrix, size_t rows, size_t cols);
int convolve_rows(const Matrix *source,
                  const Matrix *kernel,
                  Matrix *destination,
                  size_t first_row,
                  size_t end_row);

#endif
```

Листинг А.3 – Файл matrix.c

```
#include "matrix.h"
#include <errno.h>
#include <math.h>
```

```
#include <stdint.h>
#include <stdlib.h>

int matrix_create(Matrix *matrix, size_t rows, size_t cols) {
    if (rows == 0 || cols == 0) {
        errno = EINVAL;
        return -1;
    }
    if (cols > SIZE_MAX / rows || rows * cols > PTRDIFF_MAX / sizeof(double))
    ↪  {
        errno = EOVERFLOW;
        return -1;
    }
    double *data = malloc(rows * cols * sizeof(*data));
    if (data == NULL) {
        return -1;
    }
    *matrix = (Matrix){rows, cols, data};
    return 0;
}

int convolve_rows(const Matrix *source,
                  const Matrix *kernel,
                  Matrix *destination,
                  size_t first_row,
                  size_t end_row) {
    ptrdiff_t center = (ptrdiff_t)(kernel->rows / 2);
    for (size_t row = first_row; row < end_row; ++row) {
        for (size_t col = 0; col < source->cols; ++col) {
            double sum = 0.0;
            for (size_t u = 0; u < kernel->rows; ++u) {
                ptrdiff_t y = (ptrdiff_t)row + center - (ptrdiff_t)u;
                if (y < 0 || y >= (ptrdiff_t)source->rows) {
                    continue;
                }
                for (size_t v = 0; v < kernel->cols; ++v) {
                    ptrdiff_t x = (ptrdiff_t)col + center - (ptrdiff_t)v;
                    if (x < 0 || x >= (ptrdiff_t)source->cols) {
                        continue;
                    }
                    sum += source->data[(size_t)y * source->cols + (size_t)x]
                    ↪  *
                           kernel->data[u * kernel->cols + v];
                }
            }
            if (!isfinite(sum)) {
                return -1;
            }
            destination->data[row * source->cols + col] = sum;
        }
    }
```

```
    return 0;
}
```

Листинг А.4 – Файл filter.h

```
#ifndef FILTER_H
#define FILTER_H

#include "matrix.h"

int apply_filter(Matrix *matrix, const Matrix *kernel, size_t iterations,
↪  size_t threads);

#endif
```

Листинг А.5 – Файл filter.c

```
#include "filter.h"
#include <errno.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    const Matrix *source;
    const Matrix *kernel;
    Matrix *destination;
    size_t first_row;
    size_t end_row;
    int result;
    pthread_t thread;
} RowTask;

static void *run_rows(void *argument) {
    RowTask *task = argument;
    task->result = convolve_rows(
        task->source, task->kernel, task->destination, task->first_row,
        ↪  task->end_row);
    return NULL;
}

static int join_workers(RowTask *tasks, size_t started) {
    int result = 0;
    for (size_t i = 0; i < started; ++i) {
        int error = pthread_join(tasks[i].thread, NULL);
        if (error != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(error));
            _Exit(EXIT_FAILURE);
        }
        if (tasks[i].result != 0) {
```

```
            result = -1;
        }
    }
    return result;
}

static int start_workers(RowTask *tasks, size_t count, RowTask *main_task,
↪  size_t *started) {
    size_t chunk = main_task->source->rows / count;
    size_t remainder = main_task->source->rows % count;
    for (size_t i = 0; i < count - 1; ++i) {
        size_t end = main_task->first_row + chunk + (i < remainder);
        tasks[i] = *main_task;
        tasks[i].end_row = end;
        int error = pthread_create(&tasks[i].thread, NULL, run_rows,
        ↪  &tasks[i]);
        if (error != 0) {
            return error;
        }
        ++*started;
        main_task->first_row = end;
    }
    return 0;
}

static int run_pass(const Matrix *source, const Matrix *kernel, Matrix
↪  *destination, size_t limit) {
    size_t count = limit < source->rows ? limit : source->rows;
    RowTask main_task = {
        .source = source, .kernel = kernel, .destination = destination,
        ↪  .end_row = source->rows};
    RowTask *tasks = NULL;
    if (count > 1) {
        if (count - 1 > SIZE_MAX / sizeof(*tasks)) {
            errno = EOVERFLOW;
            return -1;
        }
        tasks = calloc(count - 1, sizeof(*tasks));
        if (tasks == NULL) {
            return -1;
        }
    }
    size_t started = 0;
    int error = start_workers(tasks, count, &main_task, &started);
    if (error == 0) {
        run_rows(&main_task);
    }
    int result = join_workers(tasks, started);
    free(tasks);
    if (error != 0 || result != 0 || main_task.result != 0) {
        errno = error != 0 ? error : ERANGE;
```

```
        return -1;
    }
    return 0;
}

static int is_filter_input_valid(const Matrix *matrix,
                                 const Matrix *kernel,
                                 size_t iterations,
                                 size_t threads) {
    if (matrix->data == NULL || kernel->data == NULL) {
        return 0;
    }
    if (iterations == 0 || threads == 0) {
        return 0;
    }
    return kernel->rows > 0 && kernel->rows == kernel->cols && kernel->rows %
    ↪  2 == 1;
}

int apply_filter(Matrix *matrix, const Matrix *kernel, size_t iterations,
↪  size_t threads) {
    if (!is_filter_input_valid(matrix, kernel, iterations, threads)) {
        errno = EINVAL;
        return -1;
    }
    Matrix scratch = {0};
    if (matrix_create(&scratch, matrix->rows, matrix->cols) == -1) {
        return -1;
    }
    for (size_t i = 0; i < iterations; ++i) {
        if (run_pass(matrix, kernel, &scratch, threads) == -1) {
            free(scratch.data);
            return -1;
        }
        double *previous = matrix->data;
        matrix->data = scratch.data;
        scratch.data = previous;
    }
    free(scratch.data);
    return 0;
}
```

Листинг А.6 – Файл console.h

```
#ifndef CONSOLE_H
#define CONSOLE_H

#include "matrix.h"

int parse_options(int argc, char **argv, size_t *threads);
int read_problem(Matrix *matrix, Matrix *kernel, size_t *iterations);
int print_matrix(const Matrix *matrix);
```

```
#endif
```

Листинг А.7 – Файл console.c

```
#include "console.h"
#include <ctype.h>
#include <errno.h>
#include <inttypes.h>
#include <limits.h>
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef struct {
    char *line;
    size_t capacity;
    int interactive;
} Reader;

typedef enum {
    INPUT_VALID,
    INPUT_INVALID,
    INPUT_TOO_LARGE
} InputResult;

static void skip_space(char **cursor) {
    while (isspace((unsigned char)**cursor)) {
        ++*cursor;
    }
}

static InputResult parse_size(char **cursor, size_t *value) {
    skip_space(cursor);
    if (!isdigit((unsigned char)**cursor)) {
        return INPUT_INVALID;
    }

    errno = 0;
    char *end;
    uintmax_t parsed = strtoumax(*cursor, &end, 10);
    if (*end != '\0' && !isspace((unsigned char)*end)) {
        return INPUT_INVALID;
    }
    if (errno == ERANGE || parsed > SIZE_MAX) {
        return INPUT_TOO_LARGE;
    }
    if (errno != 0 || parsed == 0) {
        return INPUT_INVALID;
```

```
    }

    *cursor = end;
    *value = (size_t)parsed;
    return INPUT_VALID;
}

static void print_size_error(InputResult result, const char *invalid_message)
↪  {
    if (result == INPUT_TOO_LARGE) {
        fprintf(stderr, "Число слишком большое. Максимальное значение:
        ↪  %zu.\n", SIZE_MAX);
    } else {
        fputs(invalid_message, stderr);
    }
}

static int at_end(char *cursor) {
    skip_space(&cursor);
    return *cursor == '\0';
}

static int read_line(Reader *reader, const char *prompt) {
    if (reader->interactive) {
        fputs(prompt, stderr);
        fflush(stderr);
    }
    if (getline(&reader->line, &reader->capacity, stdin) >= 0) {
        return 0;
    }
    if (ferror(stdin)) {
        perror("Чтение ввода");
    } else {
        fputs("Ввод закончился раньше, чем получены все данные.\n", stderr);
    }
    return -1;
}

static int read_dimensions(Reader *reader, size_t *rows, size_t *cols) {
    do {
        if (read_line(reader, "Число строк и столбцов (например, 2 3): ") ==
        ↪  -1) {
            return -1;
        }
        char *cursor = reader->line;
        InputResult result = parse_size(&cursor, rows);
        if (result == INPUT_VALID) {
            result = parse_size(&cursor, cols);
        }
        if (result == INPUT_VALID && at_end(cursor)) {
            return 0;
```

```
        }
        print_size_error(result, "Нужны два положительных целых числа через
        ↪  пробел.\n");
    } while (reader->interactive);
    return -1;
}

static int read_size(Reader *reader, const char *prompt, size_t *value, int
↪  odd) {
    do {
        if (read_line(reader, prompt) == -1) {
            return -1;
        }
        char *cursor = reader->line;
        InputResult result = parse_size(&cursor, value);
        if (result == INPUT_VALID && at_end(cursor) && (!odd || *value % 2 ==
        ↪  1)) {
            return 0;
        }
        print_size_error(result, odd ? "Нужно положительное нечётное целое
        ↪  число.\n"
                                     : "Нужно положительное целое число.\n");
    } while (reader->interactive);
    return -1;
}

static InputResult parse_row(char *cursor, double *values, size_t count) {
    for (size_t i = 0; i < count; ++i) {
        skip_space(&cursor);
        errno = 0;
        char *end;
        double value = strtod(cursor, &end);
        if (errno == ERANGE && isinf(value)) {
            return INPUT_TOO_LARGE;
        }
        if (end == cursor || !isfinite(value)) {
            return INPUT_INVALID;
        }
        if (errno != 0 && errno != ERANGE) {
            return INPUT_INVALID;
        }
        if (errno == ERANGE && value == 0.0) {
            return INPUT_INVALID;
        }
        if (*end != '\0' && !isspace((unsigned char)*end)) {
            return INPUT_INVALID;
        }
        if ((long double)fabs(value) > (long double)ULLONG_MAX) {
            return INPUT_TOO_LARGE;
        }
        values[i] = value;
```

```
        cursor = end;
    }
    return at_end(cursor) ? INPUT_VALID : INPUT_INVALID;
}

static int read_matrix(Reader *reader, Matrix *matrix, const char *name) {
    for (size_t row = 0; row < matrix->rows; ++row) {
        for (;;) {
            if (reader->interactive) {
                fprintf(stderr,
                        "%s, строка %zu/%zu (%zu чисел): ",
                        name,
                        row + 1,
                        matrix->rows,
                        matrix->cols);
            }
            if (read_line(reader, "") == -1) {
                return -1;
            }
            InputResult result = parse_row(
                reader->line, matrix->data + row * matrix->cols,
                ↪  matrix->cols);
            if (result == INPUT_VALID) {
                break;
            }
            if (result == INPUT_TOO_LARGE) {
                fprintf(stderr, "Модуль числа слишком большой. Максимум:
                ↪  %llu. "
                        "Повторите строку.\n", ULLONG_MAX);
            } else {
                fprintf(stderr, "Введите ровно %zu конечных чисел через
                ↪  пробел; "
                        "дробная часть через точку. Повторите строку.\n",
                        ↪  matrix->cols);
            }
            if (!reader->interactive) {
                return -1;
            }
        }
    }
    return 0;
}

int parse_options(int argc, char **argv, size_t *threads) {
    if (argc == 2 && strcmp(argv[1], "--help") == 0) {
        puts("Свёртка матрицы, лабораторная работа 3, вариант 13.\n"
             "Запуск: ./convolution [--threads P]\n"
             "P — максимум потоков вместе с главным; по умолчанию 1.\n"
             "Ввод по строкам:\n"
             "  строки столбцы\n"
             "  значения каждой строки матрицы\n"
```

```
            "  нечётный размер квадратного ядра\n"
            "  значения каждой строки ядра\n"
            "  число проходов K\n"
            "Размеры и K положительные; числа конечные, дробная часть через
             ↪  точку.\n"
            "Модуль элементов матрицы и ядра не больше ULLONG_MAX.\n"
            "За границами матрицы нули. Ядро переворачивается (свёртка).\n"
            "Можно перенаправить ввод: ./convolution --threads 4 <
             ↪  input.txt");
        return 1;
    }
    *threads = 1;
    if (argc == 1) {
        return 0;
    }
    if (argc == 3 && strcmp(argv[1], "--threads") == 0) {
        char *cursor = argv[2];
        InputResult result = parse_size(&cursor, threads);
        if (result == INPUT_VALID && at_end(cursor)) {
            return 0;
        }
        print_size_error(result, "Неверный лимит потоков. Нужно положительное
         ↪  целое число.\n");
        return -1;
    }
    fputs("Неверные параметры. Используйте ./convolution --help.\n", stderr);
    return -1;
}

static int read_input(Reader *reader, Matrix *matrix, Matrix *kernel, size_t
 ↪  *iterations) {
    size_t rows, cols, size;
    if (read_dimensions(reader, &rows, &cols) == -1) {
        return -1;
    }
    if (matrix_create(matrix, rows, cols) == -1) {
        perror("Выделение матрицы");
        return -1;
    }
    if (read_matrix(reader, matrix, "Матрица") == -1) {
        return -1;
    }
    if (read_size(reader, "Размер ядра (нечётное число, например 3): ",
     ↪  &size, 1) == -1) {
        return -1;
    }
    if (matrix_create(kernel, size, size) == -1) {
        perror("Выделение ядра");
        return -1;
    }
    if (read_matrix(reader, kernel, "Ядро") == -1) {
```

```
        return -1;
    }
    return read_size(reader, "Количество проходов K: ", iterations, 0);
}

int read_problem(Matrix *matrix, Matrix *kernel, size_t *iterations) {
    Reader reader = {.interactive = isatty(STDIN_FILENO)};
    if (reader.interactive) {
        fputs("Введите матрицу и ядро построчно. Числа разделяйте пробелами,
        ↪  "
              "дробную часть — точкой.\n",
              stderr);
    }
    int result = read_input(&reader, matrix, kernel, iterations);
    free(reader.line);
    return result;
}

int print_matrix(const Matrix *matrix) {
    if (isatty(STDIN_FILENO)) {
        fputs("Результат:\n", stderr);
    }
    for (size_t row = 0; row < matrix->rows; ++row) {
        for (size_t col = 0; col < matrix->cols; ++col) {
            if (printf("%s%.17g", col == 0 ? "" : " ", matrix->data[row *
            ↪  matrix->cols + col]) <
                0) {
                return -1;
            }
        }
        if (putchar('\n') == EOF) {
            return -1;
        }
    }
    return fflush(stdout) == EOF ? -1 : 0;
}
```